

Formalização da Correção do Algoritmo de Ordenação por Inserção Binária

Daniel da Cunha Pereira Luz – 211055540
Thiago Veras Rodrigues Queiroz – 211055370
Vinícius Bowen – 180079239

15 de julho de 2026

1 Introdução

Este relatório documenta a formalização, no assistente de provas Rocq, da correção do algoritmo de ordenação por inserção binária (*binary insertion sort*). O algoritmo, implementado pela função `bininsertion_sort`, insere cada elemento de uma lista em sua posição correta dentro de uma sublista já ordenada, usando uma busca binária (`bsearch`) para localizar essa posição em tempo logarítmico, em vez da busca linear usada na inserção tradicional.

Formalmente, a correção do algoritmo é estabelecida por dois resultados combinados: que a lista produzida por `bininsertion_sort` está ordenada (`Sorted le`) e que ela é uma permutação da lista de entrada (`Permutation`). O código-fonte completo com todas as definições e provas está disponível no repositório do projeto; neste documento, cada integrante descreve em linguagem natural a estratégia de prova da parte sob sua responsabilidade.

2 Fundamentos e Propriedades Estruturais

Esta seção detalha o desenvolvimento dos blocos de construção fundamentais do projeto, sob a responsabilidade do integrante Daniel, localizados no arquivo `src/bininsertion_sort.v`. Esses blocos garantem a validade matemática dos limites das buscas e descrevem as propriedades estruturais de manipulação de listas ao realizar uma inserção direta.

Para fornecer uma fundamentação formal sólida alinhada aos conceitos da disciplina de Lógica Computacional 1, a especificação e as provas baseiam-se em estruturas indutivas descritas por gramáticas formais. O tipo de dados para os números naturais ($\mathbb{N}$) e as listas de naturais (`list  $\mathbb{N}$`) são definidos pelas seguintes regras de formação:

$\frac{\text{NAT_BASE}}{0 \in \mathbb{N}}$	$\frac{\text{NAT_REC} \quad n \in \mathbb{N}}{S(n) \in \mathbb{N}}$	$\frac{\text{LIST_BASE}}{\text{nil} \in \text{list } \mathbb{N}}$	$\frac{\text{LIST_REC} \quad x \in \mathbb{N} \quad \ell \in \text{list } \mathbb{N}}{\text{cons}(x, \ell) \in \text{list } \mathbb{N}}$
---	--	--	---

2.1 Testes de Execução Computacional

Antes de iniciar as demonstrações de teoremas gerais, foram definidos cinco lemas de testes computacionais (`test0` a `test4`). Esses testes realizam a avaliação simbólica e a redução de expressões concretas no interpretador do Rocq. A estratégia de prova desses testes baseia-se na reescrita da equação recursiva da busca binária (`rewrite bsearch_equation`) seguida pela simplificação direta (`simpl`) e reflexividade de igualdade (`reflexivity`).

Do ponto de vista lógico-computacional, esse processo de redução baseia-se no **Princípio de Poincaré** (ou conversão por avaliação), que garante a equivalência entre computação sintática e demonstração lógica na teoria de tipos do Rocq. A $\beta\eta\iota$ -redução simplifica termos fechados por computação direta no interpretador até que se alcance uma identidade sintática de termos, fechando o objetivo por reflexividade (`refl_equal`).

No caso de testes que envolvem comparações aritméticas em ramificações falsas de hipóteses absurdas (por exemplo, quando assumimos uma hipótese contraditória como $1 \leq 0$), utilizou-se a inversão de hipóteses absurdas (**inversion**). Em Dedução Natural, a regra de eliminação do absurdo ($\perp_e$) estabelece que de uma contradição pode-se derivar qualquer proposição. A tática **inversion** analisa as regras de formação do predicado indutivo aritmético e detecta que nenhuma regra pode construir o termo de desigualdade absurda, descarregando o ramo correspondente e garantindo a sanidade computacional de **bsearch**.

2.2 Abordagem de Prova: Limites da Busca (**bsearch_valid_pos**)

A função **bsearch** calcula o índice ideal de inserção dividindo recursivamente a lista original ao meio usando as funções auxiliares **firstn** (prefixo) e **skipn** (sufixo). O lema **bsearch_valid_pos** estabelece que o índice retornado por essa busca sempre respeita os limites físicos da lista, isto é, está no intervalo fechado entre 0 e o tamanho total da lista.

$$\frac{\text{BSEARCH_BOUNDS} \quad x \in \mathbb{N} \quad \ell \in \text{list } \mathbb{N}}{0 \leq \text{bsearch } x \ell \leq \text{length}(\ell)}$$

Como a redução do tamanho da lista em **bsearch** não é linear (ou seja, o algoritmo divide a lista em metades e não decreta elemento por elemento), o princípio clássico de indução estrutural sobre a lista falha. Para contornar essa barreira, a demonstração adota o princípio de **indução funcional** (**functional induction**).

A indução funcional constrói um princípio de indução personalizado com base no grafo de chamadas recursivas da função **bsearch**. Se definirmos a propriedade a ser provada como $P(x, \ell, \text{bsearch}(x, \ell))$, a indução funcional gera hipóteses de indução estruturadas de acordo com as ramificações de execução do algoritmo:

1. **Caso Base 1 (Lista Vazia):** Se a lista for vazia ($\ell = []$), o algoritmo retorna diretamente 0. Como $\text{length}([]) = 0$, o limite $0 \leq 0 \leq 0$ é imediato por reflexividade aritmética.
2. **Caso Base 2 (Lista Unitária):** Se a lista contém apenas um elemento ($\ell = [y]$), o algoritmo testa se $x \leq y$. Se verdadeiro, retorna 0; se falso, retorna 1. Em ambos os caminhos, o resultado é limitado pelo tamanho total da lista unitária (1).
3. **Caso Geral 1 (Subdivisão à Esquerda):** Quando o elemento procurado é menor do que o ponto médio ($x < h'_2$), o algoritmo recorre sobre o prefixo da lista obtido via **firstn** de tamanho `mid`. Pela regra de eliminação do quantificador universal ($\forall_e$), a hipótese de indução é instanciada para a sublista da esquerda:

$$H_i : 0 \leq \text{bsearch } x \text{ (firstn mid } \ell) \leq \text{length}(\text{firstn mid } \ell)$$

Para estender essa propriedade à lista original, a prova reescreve a relação estrutural de tamanho usando o lema aritmético **length_firstn**, que estabelece:

$$\text{length}(\text{firstn mid } \ell) = \min(\text{mid}, \text{length}(\ell))$$

Como $\text{mid} = \text{length}(\ell)/2$, as relações de desigualdade linear são unificadas e solucionadas de forma puramente aritmética pelo resolvidor linear (**lia**), que implementa o algoritmo de eliminação de Fourier-Motzkin para a aritmética de Presburger.

4. **Caso Geral 2 (Ponto Médio):** Se o elemento procurado coincide com o ponto médio da lista ($x = h'_2$), a função encerra a recursão e retorna diretamente o índice médio mid. Como a lista possui comprimento maior ou igual a 2, provar que $0 \leq \text{mid} \leq \text{length}(\ell)$ decorre de propriedades de divisão inteira por 2, auxiliadas pelo teorema `Nat.div_lt`.
5. **Caso Geral 3 (Subdivisão à Direita):** Quando o elemento procurado é maior do que o ponto médio ($x > h'_2$), o algoritmo recorre sobre o sufixo obtido via `skipn`, retornando a soma do índice médio mid com o resultado da chamada recursiva. A hipótese de indução garante a validade do índice na sublista:

$$H_i : 0 \leq \text{bsearch } x \text{ (skipn mid } \ell) \leq \text{length}(\text{skipn mid } \ell)$$

O limite global superior é deduzido utilizando o lema `length_skipn`, que relaciona o tamanho do sufixo com a diferença aritmética:

$$\text{length}(\text{skipn mid } \ell) = \text{length}(\ell) - \text{mid}$$

Substituindo essa identidade e somando mid em todos os lados da desigualdade da hipótese de indução, o resolvidor linear (`lia`) unifica os termos e fecha o objetivo demonstrando que a soma das partições preserva estritamente o limite superior original.

2.3 Abordagem de Prova: Propriedades Estruturais de `insert_at`

A função `insert_at i x ℓ` insere o elemento x no índice i da lista ℓ . As demonstrações das propriedades essenciais dessa função basearam-se no princípio de indução estrutural clássico sobre a lista ℓ , cuja especificação lógica corresponde ao seguinte axioma de segunda ordem:

$$\forall P, (P(\[]) \wedge \forall y \forall \ell', (P(\ell') \rightarrow P(y :: \ell'))) \rightarrow \forall \ell, P(\ell)$$

- **Prova de Conservação de Tamanho (`insert_at_length`):** O lema garante que a inserção incrementa o comprimento da lista em exatamente uma unidade.

$$\frac{\text{INSERT_LENGTH} \quad x, i \in \mathbb{N} \quad \ell \in \text{list } \mathbb{N}}{\text{length}(\text{insert_at } i \ x \ \ell) = S(\text{length}(\ell))}$$

A abordagem de prova seguiu por indução estrutural sobre a lista ℓ , combinada com análise de casos sob o índice de destino i utilizando as regras do quantificador universal ($\forall_i$) e ($\forall_e$):

1. **Caso Base ($\ell = []$):** Queremos mostrar $\forall i \forall x, \text{length}(\text{insert_at } i \ x \ []) = S(\text{length}([])) = S(0)$. Fazendo análise de casos sob $i \in \mathbb{N}$: - Se $i = 0$, então por definição `insert_at 0 x [] = [x]`, cujo comprimento é $1 = S(0)$. - Se $i = S(k)$, então por definição `insert_at (S(k)) x [] = [x]`, cujo comprimento é $1 = S(0)$. Ambos os ramos são triviais por definição e reflexividade.
2. **Caso Indutivo ($\ell = h :: t$):** Assumimos a hipótese de indução para a cauda da lista:

$$H_i : \forall i \forall x, \text{length}(\text{insert_at } i \ x \ t) = S(\text{length}(t))$$

Queremos provar que:

$$\forall i \forall x, \text{length}(\text{insert_at } i \ x \ (h :: t)) = S(\text{length}(h :: t)) = S(S(\text{length}(t)))$$

Fazendo análise de casos sobre o índice i : - Se $i = 0$, temos por definição que `insert_at 0 x (h :: t) = x :: h :: t`, cujo tamanho é $S(\text{length}(h :: t))$, o que fecha por reflexividade. - Se $i = S(k)$, a expressão por definição de recursão de `'insert_at'` torna-se `h :: insert_at k x t`. O tamanho dessa lista é $S(\text{length}(\text{insert_at } k \ x \ t))$. Pela regra de congruência funcional de igualdades (tática `f_equal`):

$$\frac{\text{length}(\text{insert_at } k \ x \ t) = S(\text{length}(t))}{S(\text{length}(\text{insert_at } k \ x \ t)) = S(S(\text{length}(t)))} \text{ (congruência)}$$

A premissa da regra de congruência é exatamente a hipótese de indução desprovida de quantificadores através de ($\forall_e$) instanciada para k e x . Logo, o objetivo é fechado de forma limpa.

- **Prova de Preservação de Conteúdo (`insert_at_perm`):** Demonstra que a lista resultante após a inserção é uma permutação válida da lista de entrada com o novo elemento anexado à cabeça ($x :: \ell$). A prova seguiu por indução estrutural na lista ℓ . O caso base e o subcaso de inserção na cabeça da lista ($i = 0$) são imediatos por reflexividade da relação de permutação. No caso indutivo geral ($\ell = h :: t$), onde o elemento é inserido na cauda recursivamente ($i = S(k)$), a hipótese de indução garante que a sublista alterada é uma permutação de sua forma com x anexado:

$$H_{ih} : \text{Permutation}(\text{insert_at } k \ x \ t, \ x :: t)$$

A partir dessa premissa, a prova aplicou a regra de congruência de cabeça de permutações (`perm_skip`) para obter:

$$\frac{\text{PERM_SKIP} \quad \text{Permutation}(\text{insert_at } k \ x \ t, \ x :: t)}{\text{Permutation}(h :: \text{insert_at } k \ x \ t, \ h :: x :: t)}$$

Em seguida, utilizou-se a regra de simetria de vizinhos adjacentes (`perm_swap`) para realizar a transposição de cabeça:

$$\frac{\text{PERM_SWAP}}{\text{Permutation}(h :: x :: t, \ x :: h :: t)}$$

Por fim, aplicando a regra de transitividade de permutações (`perm_trans`) sobre os dois passos dedutivos anteriores, o Rocq pôde unificar as pontas de equivalência e fechar o teorema provando que a inserção em qualquer posição intermediária preserva o conjunto de elementos original de forma rigorosa:

$$\frac{\text{PERM_TRANS} \quad \text{Permutation}(h :: \text{insert_at } k \ x \ t, \ h :: x :: t) \quad \text{Permutation}(h :: x :: t, \ x :: h :: t)}{\text{Permutation}(h :: \text{insert_at } k \ x \ t, \ x :: h :: t)}$$

2.4 Desafios e Adaptações de Automação no Rocq 9

Durante o desenvolvimento no Rocq 9, o principal desafio residiu na instabilidade da tática de indução funcional ao lidar com nomes dinâmicos de variáveis gerados pelo compilador (como sublistas intermediárias). Scripts de prova que citavam variáveis diretamente falhavam em atualizações do ambiente do assistente. Para contornar essa fragilidade e tornar as demonstrações robustas, o script de `bsearch_valid_pos` foi reestruturado para ser executado em lote por meio do combinador `all:`, aplicando táticas baseadas unicamente em estruturas de tipos (como `length_firstn` e `length_skipn`) e permitindo que o resolvidor aritmético resolvesse todos os ramos recursivos simultaneamente sem apelo a nomes dinâmicos. Adicionalmente, na prova de tamanho, a hipótese de indução apresentava-se sob uma quantificação universal aberta (`forall i x`), o que impedia sua aplicação direta pelo resolvidor automático. Isso foi solucionado pela introdução da tática de desestruturação funcional `f_equal` para remover construtores externos antes da unificação dos tipos.

O desenvolvimento bem-sucedido dessas propriedades forneceu ao Thiago e ao Vinícius as garantias de estabilidade de índices e invariantes de permutações necessárias para a prova dos teoremas de ordenação global do projeto.

3 Correção de `binsert`

Nesta seção é apresentada a prova de que a função `binsert`, responsável por inserir um elemento `x` em uma lista ordenada `l` na posição retornada por `bsearch`, preserva a ordenação da lista. Em outras palavras, mostra-se que, se `l` está ordenada, então `insert_at (bsearch x l) x l` também estará.

3.1 Estratégia geral

A demonstração foi dividida em duas etapas principais.

1. Primeiro, foi provado um resultado sobre `insert_at`, independente da função `bsearch`. A ideia é mostrar que, se a posição de inserção separa corretamente a lista — isto é, todos os elementos anteriores são menores ou iguais a `x`, enquanto todos os elementos a partir dessa posição são maiores ou iguais a `x` — então a inserção preserva a ordenação da lista.
2. Em seguida, foi demonstrado que a posição retornada por `bsearch` satisfaz exatamente essa propriedade sempre que a lista de entrada já estiver ordenada.

A combinação desses dois resultados permite concluir diretamente o teorema principal, uma vez que `binsert x l` é definido como `insert_at (bsearch x l) x l`.

3.2 Lemas auxiliares sobre listas ordenadas

Antes da prova principal, foi necessário estabelecer alguns resultados auxiliares sobre listas ordenadas que não estavam disponíveis na biblioteca padrão na forma necessária para esta formalização.

- Em uma lista ordenada `a :: l`, o elemento `a` é menor ou igual a todos os elementos de `l`, e não apenas ao primeiro.
- O prefixo (`firstn`) de uma lista ordenada também permanece ordenado.
- O sufixo (`skipn`) de uma lista ordenada também permanece ordenado.
- Em uma lista escrita como `P ++ a :: Q`, todos os elementos de `P` são menores ou iguais a `a`.

Embora esses resultados sejam esperados para listas ordenadas, foi necessário demonstrá-los explicitamente por indução, já que a definição de `Sorted` fornece apenas propriedades entre elementos consecutivos.

3.3 Ligando `bsearch` à ordenação

O principal resultado desta etapa mostra que a posição retornada por `bsearch` realmente divide a lista em dois grupos: antes da posição ficam apenas elementos menores ou iguais a `x`, enquanto a partir dela ficam apenas elementos maiores ou iguais.

A demonstração foi feita por indução funcional sobre a definição de `bsearch`, acompanhando seus casos: lista vazia, lista unitária e lista dividida ao meio.

O caso de maior complexidade ocorre quando a lista é particionada em duas metades. Nessa situação, `bsearch` realiza uma chamada recursiva sobre apenas uma das metades, e a posição final é obtida somando o deslocamento correspondente ao ponto médio. A prova consiste em mostrar que essa posição continua separando corretamente a lista original. Para isso, foi utilizada a hipótese de indução sobre a metade escolhida em conjunto com os lemas sobre listas ordenadas apresentados anteriormente.

3.4 Uma dificuldade encontrada

Durante o desenvolvimento dessa prova foi necessário alterar a estratégia utilizada no caso em que a busca continua na metade direita da lista.

Inicialmente, a decomposição da lista era feita por meio de reescritas com `replace ... at N`, escolhendo manualmente qual ocorrência deveria ser substituída. Essa abordagem acabou se mostrando pouco robusta, pois o uso de `set` para nomear a lista e o ponto médio fazia com que a numeração das ocorrências variasse durante a prova, tornando a aplicação da tática instável.

A solução adotada foi substituir essa abordagem por dois lemas gerais sobre `firstn` e `skipn`, relacionando diretamente índices da forma `n + k` com aplicações sucessivas dessas funções. Dessa forma, a prova deixou de depender da posição das ocorrências na expressão e passou a utilizar apenas propriedades estruturais das listas, tornando-se mais simples e estável.

3.5 Fechando a prova de `binsert_correct`

Após estabelecer o lema de separação e o resultado sobre `insert_at`, a demonstração do teorema `binsert_correct` torna-se direta. Basta aplicar o lema de separação à posição retornada por `bsearch` e utilizar `bsearch_valid_pos` para garantir que essa posição pertence aos limites da lista.

4 Conclusão e Prova do Teorema Final

O teorema principal do projeto, `binsertion_sort_correct`, consolida o trabalho estabelecendo que, para qualquer lista de entrada l , o algoritmo `binsertion_sort` retorna uma lista simultaneamente ordenada e que consiste em uma permutação exata da lista original[cite: 1].

A demonstração deste teorema é construída de forma direta por indução estrutural sobre a lista l , o que divide a estratégia em dois objetivos de prova principais[cite: 2]:

- **Caso Base (Lista Vazia):** O primeiro objetivo avalia a correção do algoritmo para uma lista sem elementos (`[]`)[cite: 2].
 - Inicialmente, utiliza-se a tática `split` para separar a conjunção lógica ($\wedge$), permitindo o tratamento da ordenação e da permutação de forma isolada[cite: 2].
 - A primeira ramificação da árvore requer a prova de que a lista vazia está ordenada. Esse passo é solucionado trivialmente pela aplicação do axioma `Sorted_nil` da biblioteca padrão `Sorted`[cite: 2].
 - A segunda ramificação exige provar que a lista vazia é uma permutação de si mesma. O objetivo é provado imediatamente pelo termo de reflexividade `Permutation_refl` da biblioteca `Permutation`[cite: 2].
- **Passo Indutivo (Lista com Elementos):** O segundo objetivo assume como premissa a hipótese de indução para a cauda da lista (`tl`) e busca demonstrar a validade do teorema para a lista conectada (`h :: tl`)[cite: 2].
 - O primeiro passo consiste em desestruturar a hipótese de indução por meio da tática `destruct`, dividindo-a explicitamente em duas garantias: a de ordenação (`Hsorted`) e a de permutação (`Hperm`) da sublista[cite: 2].
 - Aplica-se novamente a tática `split` para bifurcar o objetivo atual[cite: 2].
 - A primeira ramificação exige demonstrar o predicado de ordenação: `Sorted le (binsertion_sort (h :: tl))`[cite: 2]. A solução é alcançada aplicando o lema central `binsert_correct` (demonstrado na seção anterior), repassando a hipótese de que a sublista já está ordenada[cite: 1, 2].
 - A segunda ramificação busca comprovar a invariância estrutural. Para fechar a prova, basta aplicar o lema auxiliar de permutação `binsertion_sort_Perm`, estruturado unicamente para esse propósito[cite: 2].

Com o fechamento livre de falhas dessas ramificações, a prova do teorema final é concluída. Isso demonstra que a formalização em Dedução Natural integra com sucesso os limites da busca binária e as propriedades de inserção, garantindo matematicamente que a função `binsertion_sort` cumpre as especificações de ordenação integral do projeto[cite: 1].

Para ilustrar a estrutura lógica descrita, a árvore de demonstração em Dedução Natural do teorema final é formalizada a seguir:

$$\begin{array}{c}
\wedge_I \text{ (BASE)} \\
\text{SORTED_NIL} \qquad \qquad \qquad \text{PERM_REFL} \\
\hline
\frac{\text{Sorted le (binsertion_sort [])} \quad \text{Permutation [] (binsertion_sort [])}}{\text{Sorted le (binsertion_sort [])} \wedge \text{Permutation [] (binsertion_sort [])}}
\end{array}$$

$$\begin{array}{c}
\wedge_I \text{ (INDUTIVO)} \\
\text{BINSERT_CORRECT} \qquad \qquad \qquad \text{BINSERT_PERM} \\
\frac{\text{Sorted le (binsertion_sort tl)}}{\text{Sorted le (binsertion_sort (h :: tl))}} \quad \frac{\text{Permutation tl (binsertion_sort tl)}}{\text{Permutation (h :: tl) (binsertion_sort (h :: tl))}} \\
\hline
\text{Sorted le (binsertion_sort (h :: tl))} \wedge \text{Permutation (h :: tl) (binsertion_sort (h :: tl))}
\end{array}$$

$$\begin{array}{c}
\text{LIST_IND} \\
\text{Caso Base} \quad \text{Passo Indutivo} \\
\hline
\forall l, \text{Sorted le (binsertion_sort l)} \wedge \text{Permutation l (binsertion_sort l)}
\end{array}$$